Improving Neural Architecture Search by Minimizing Worst-Case Validation Loss

Haochen Zhang, Xuefeng Du, Ruisi Zhang, and Pengtao Xie, *Member, IEEE*

***Abstract*—Neural architecture search (NAS) aims at automatically searching for high-performance architectures and has achieved considerable progress. Existing NAS methods learn architectures by minimizing average-case validation losses. As a result, the searched architectures are less capable of making correct predictions under worst-case scenarios. To address this problem, we propose a framework that leverages a deep generative model to generate adversarial validation examples to measure the worst-case validation performance of an architecture and improves the architecture by minimizing the loss on the generated adversarial validation data. Our framework is based on multilevel optimization, which performs multiple learning stages end-to-end. Experiments on a variety of datasets demonstrate the effectiveness of our method.**

***Impact Statement*—One potential negative societal impact of our work is: if the generated validation examples have low fidelity, architectures searched based on these examples may make unexpected errors, which makes decision-making in mission-critical areas such as healthcare and finance unreliable. One major limitation of this work is that it is difficult to be applied to nondifferentiable NAS methods, including those based on reinforcement learning and evolutionary algorithms.**

***Index Terms*—Multilevel optimization (MLO), neural architecture search (NAS), worst-case performance.**

I. Introduction

NEURAL architecture search (NAS) [45], [54], [82] aims at automatically searching for high-performance architectures of neural networks to reduce humans' burden in manually designing them. The research of NAS has made remarkable progress in the past few years and achieved promising results. In existing NAS methods, model weights are learned by minimizing a training loss and the architecture is learned by minimizing an averaged validation loss (AVL). The AVL, calculated by taking the mean of validation losses computed on individual examples in a fixed validation set, reflects the average-case performance of a neural architecture. An architecture learned by minimizing the AVL focuses on achieving great average-case performance, but may have poor worst-case performance [57] (empirical justification is in Section IV). Consider the evaluation of a deep neural network-based treatment recommendation system. Given a validation set where 99.99% of patients have common diseases and 0.01% of patients have rare diseases, a treatment recommendation system, which can accurately recommend treatment plans for common diseases but performs poorly on rare diseases, will have high average evaluation accuracy on this validation set. However, from the clinical perspective, such a system cannot be safely used in clinical practice since it cannot properly deal with patients with rare diseases (i.e., having poor worst-case performance). This limitation is particularly consequential in safety-critical domains such as medicine, where even a small number of misclassified rare cases can lead to serious consequences. While the previous example illustrates this in the context of treatment recommendation, similar concerns arise in diagnostic settings—for instance, models may achieve high average accuracy on chest X-rays yet fail to identify rare conditions such as pneumomediastinum or certain pediatric cancers. These failure cases are often overlooked in architecture search driven by average-case validation loss.

To address the limitation of average-case evaluation, Shu et al. [57] proposed adversarial examination, which dynamically selects a sequence of validation sets so that the performance of a model (with fixed weight parameters) evaluated on these validation sets decreases. By doing this, the model's worst-case performance (or weakness) can be identified. The notion of "adversarial" comes from the fact that the validation sets are selected to minimize the model's performance. While this work can evaluate the worst-case performance of a fixed model, it does not provide a mechanism to leverage the evaluation results to retrain the model for improving its worst-case performance.

We aim to bridge this gap and propose to generate adversarial validation examples to measure and improve the worst-case performance of a neural architecture. In our framework, there is a "learner" model and a "tester" model. The tester generates adversarial validation data using a deep generative model [17] to measure the worst-case performance of the learner. The learner updates its architecture by minimizing the loss on the generated adversarial validation data to improve its worst-case performance. Our method is based on multilevel optimization

Received 6 April 2025; revised 14 June 2025 and 13 November 2025; accepted 26 February 2026. Date of publication 5 March 2026; date of current version 31 August 2026. This work was supported in part by NSF under Grant IIS2405974; in part by NSF under Grant IIS2339216; in part by NIH under Grant R35GM157217; and in part by NIH under Grant R21GM154171. This article was recommended for publication by Associate Editor Sean Holden upon evaluation of the reviewers' comments. *(Corresponding author: Pengtao Xie.)*

Haochen Zhang, Ruisi Zhang, and Pengtao Xie are with the Department of Electrical and Computer Engineering, University of California San Diego, La Jolla, CA 92093 USA (e-mail: haz035@ucsd.edu; ruisizhang123@gmail.com; p1xie@ucsd.edu).

Xuefeng Du is with the Department of Computer Sciences, University of Wisconsin - Madison, Madison, WI 53706 USA (e-mail: xuefengdu1@gmail.com).

This article has supplementary downloadable material available at https://doi.org/10.1109/TAI.2026.3670777, provided by the authors.

Digital Object Identifier 10.1109/TAI.2026.3670777

(MLO) [56], which consists of four levels of nested optimization problems. At the first level, we train model weights of the learner with its architecture tentatively fixed. At the second level, a deep generative model (DGM) is trained. At the third level, an auxiliary model is trained using data generated by the DGM to verify the fidelity of generated data. At the fourth level, the DGM generates adversarial validation data; the learner updates its architecture by minimizing the loss on the generated data; and the DGM updates its hyperparameters by minimizing the loss of the auxiliary model on a human-labeled validation set.

The major contributions of this article are as follows.

1) We propose a general framework to evaluate and improve the worst-case performance of neural architectures in NAS.
2) We demonstrate the effectiveness of our method on various datasets.

II. Related Work

A. NAS

Existing NAS methods can be roughly grouped into three categories: 1) reinforcement learning (RL) based methods; 2) evolutionary algorithm (EA) based methods; and 3) differentiable methods. In RL-based approaches [52], [82], [83], a policy is learned to iteratively generate new architectures by maximizing a reward, which is validation accuracy. EA-based approaches [44], [54] represent architectures as individuals in a population. Individuals with high fitness scores (validation accuracy) have the privilege to generate offspring, which replaces individuals with low fitness scores. Differentiable approaches [4], [45], [68] adopt a network pruning strategy. On top of an over-parameterized network, the importance weights of operators are learned using gradient descent. Operators with close-to-zero weights are pruned. Previous NAS methods cannot improve architectures' worst-case performance. Our work aims to bridge this gap.

B. Adversarial Learning

Our formulation involves a mini-max optimization problem, which is analogous to that in adversarial learning [17] for data generation [17], [74], domain adaptation [15], adversarial attack and defense [18], active learning [49], etc. Different from existing works, our work is featured with a tester which generates adversarial validation data to minimize the validation performance of a learner while the learner retrains itself to improve performance on generated validation data. Shu et al. [57] proposed using an adversarial examiner to identify the weakness of a trained model. Our work differs from [57] in that we continuously update a learner model based on how it performs on the validation examples that are generated dynamically by a tester model, while the learner model in [57] is fixed and not affected by examination results. Such et al. [62] proposed to learn a generative adversarial network [17] to create synthetic examples which are used to train an NAS model. Our work differs from [17] in that we generate adversarial validation data to evaluate the worst-case performance of a model while Such et al. [62] do not consider the notion of "adversarial".

C. Curriculum Learning (CL)

CL has been widely studied [2], [19], [34], [39], [40], [48], [53], [61], [65], where a sequence of training datasets with increasing levels of difficulty is used for model training. Some CL works [2], [53], [61], [65], [81] use task-specific prior knowledge to measure hardness of examples, which are not generalizable across a variety of tasks. Self-paced CL [34], [35], [39], [79] automatically measures example hardness using training losses and the hardness can be dynamically adjusted. Our work differs from these previous works in that our work dynamically generates adversarial validation data for model evaluation while previous works select hard data examples for model training.

III. Method

A. Overview

In our framework, there is a learner model and a tester model (Fig. 1), where the learner learns to perform a target task such as classification, regression, etc. The tester model uses a deep generative model (DGM) [17] to generate adversarial validation examples to evaluate the worst-case performance of the learner. These validation examples are generated in a way that the learner's performance on these examples [referred to as generative adversarial validation examples (GAVEs)] decreases. The learner continuously retrains its model by maximizing the performance on GAVEs, to improve its worst-case performance. Note that GAVEs are different from adversarial examples [18] in the adversarial robustness literature. GAVEs are brand new examples that are generated by a DGM and have significant visual and semantic differences with real examples in a given dataset. In contrast, adversarial examples are created by adding small perturbations to real examples, and are very similar to real examples both visually and semantically. GAVEs focus on evaluating the worst-case performance of an ML model over the global landscape of a data distribution (which is the focus of our work), while adversarial examples focus on evaluating the robustness of an ML model around a local example. In Supplementary Section B.2, we performed an evaluation of our method on adversarial examples.

Besides decreasing the learner's validation performance, generated validation examples should be "meaningful". It is possible that generated validation examples are of poor quality, which are outliers or have incorrect class labels. Using low-quality validation examples to guide the learning of the learner may render the learner to overfit these examples. To address this problem, we evaluate the "meaningfulness" of generated examples by checking how useful they are when used to train an auxiliary model for performing the target task. If the auxiliary model trained on generated data achieves good validation performance on human-labeled real data examples, the generated examples are considered meaningful.

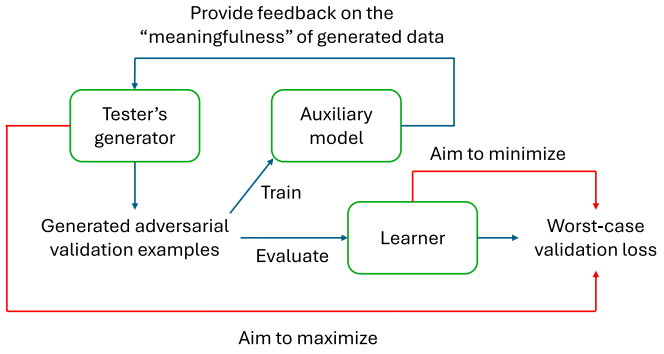


Fig. 1. Diagram showing the interaction of key components in our method.

In our framework, both the learner and the tester perform learning. The learner aims to improve its worst-case performance. The tester learns to generate validation examples that can decrease the learner's validation performance and are meaningful. The learner has a learnable architecture. Since the learner and tester perform adversarial learning jointly, it is important to let them have similar model capacities to prevent one of them from being dominated by the other. Towards this goal, we make hyperparameters of the tester learnable. In Supplementary Section B.2.2, we provided experiments which show that learning the hyperparameters of the tester yields better performance than not learning them.

B. MLO-Based Framework

In our framework, the learning of the learner and the tester is organized into four stages.

1) Stage I: Let A denote the learner's architecture and $L_{\text{tgt}}(\cdot)$ denote the target task's training loss. At this stage, the learner trains its network weights W on a training set $D^{(tr)}$

$$W^*(A) = \underset{W}{\text{argmin}}\ L_{\text{tgt}}(W, A, D^{(tr)}). \tag{1}$$

The architecture A is used to define the training loss, but it is not learned at this stage. If A is learned by minimizing this training loss, a trivial solution will be yielded where the model can perfectly overfit the training data but will generalize poorly on test data. Let $W^*(A)$ denote the optimally learned W at this stage. Note that W^* depends on A because W^* depends on the training loss and the training loss is a function of A.

2) Stage II: At the second stage, the tester trains a data generation model which generates both input data and output labels. Let G and B denote weight parameters and hyperparameters of the data generation model. Let L_{dg} denote a data generation loss. At this stage, we solve the following optimization problem:

$$G^*(B) = \underset{G}{\text{argmin}}\ L_{\text{dg}}(G, B, D^{(tr)}) \tag{2}$$

G is trained on $D^{(tr)}$ by switching the input and output in each training example and B is tentatively fixed.

3) Stage III: At the third stage, we apply the generator $G^*(B)$ trained at the second stage to generate a validation dataset $f(G^*(B))$. Then we train an auxiliary model V on $f(G^*(B))$ to perform the target task. At this stage, we solve the following optimization problem:

$$V^*(G^*(B)) = \underset{V}{\text{argmin}}\ L_{\text{tgt}}(V, f(G^*(B))). \tag{3}$$

4) Stage IV: At the fourth stage, we evaluate the learner's model $W^*(A)$ on the validation set $f(G^*(B))$. For the tester, it aims to generate an adversarial validation set by maximizing the validation loss w.r.t B. For the learner, it aims to perform well under worst-case scenarios by minimizing the loss on the generative adversarial validation set w.r.t A. On the other hand, we evaluate the auxiliary model $V^*(G^*(B))$ on a human-labeled validation set $D^{(\text{val})}$. The tester aims to minimize this loss w.r.t B to encourage the generated validation examples to be meaningful. When improving the worst-case performance of A, we would like A to maintain its average-case performance as well. We evaluate the average-case loss of A on $D^{(\text{val})}$ and update A by minimizing the average-case loss. At this stage, we solve the following optimization problem:

$$\begin{aligned}&\min_A \max_B\ L_{\text{tgt}}(W^*(A), A, f(G^*(B)))\\ &\quad - \lambda L_{\text{tgt}}(V^*(G^*(B)), D^{(\text{val})}) + \gamma L_{\text{tgt}}(W^*(A), A, D^{(\text{val})})\end{aligned} \tag{4}$$

where λ and γ are tradeoff parameters. λ strikes the balance between the worst-case-ness and meaningfulness of generated data. γ strikes the balance between worst-case and average-case performance.

5) MLO: Putting these pieces together, we have the following MLO framework:

$$\begin{aligned}&\min_A \max_B L_{\text{tgt}}(W^*(A), A, f(G^*(B)))\\ &\quad - \lambda L_{\text{tgt}}(V^*(G^*(B)), D^{(\text{val})}) + \gamma L_{\text{tgt}}(W^*(A), A, D^{(\text{val})})\\ &s.t.\ V^*(G^*(B)) = \underset{V}{\text{argmin}}\ L_{\text{tgt}}(V, f(G^*(B)))\\ &\quad G^*(B) = \underset{G}{\text{argmin}}\ L_{\text{dg}}(G, B, D^{(tr)})\\ &\quad W^*(A) = \underset{W}{\text{argmin}}\ L_{\text{tgt}}(W, A, D^{(tr)}).\end{aligned} \tag{5}$$

Table I shows a summary of each optimization stage's objective and update rule.

C. Reduce Search Costs

Before running our method on a dataset D, we first pretrain the generative model and auxiliary model on the same dataset D. The total search cost includes pretraining the generative model and auxiliary model and running our MLO-based framework to learn all variables in (9). We utilize the following approaches to reduce search and memory costs.

1) The frequencies of calculating hypergradients of architecture A, generator G, and hyperparameters B and updating these variables are reduced to every eight iterations (i.e., mini-batches) instead of every mini-batch. We empirically found that by doing this, the computational costs were substantially reduced without significantly compromising image classification performance. For the rest of parameters, they were updated in every iteration as usual.

TABLE I
SUMMARY OF EACH OPTIMIZATION STAGE'S OBJECTIVE AND UPDATE RULE

Stage	Objective	Update Rule
I	The learner trains its network weights. $W^*(A) = \text{argmin}_W\ L_{\text{tgt}}(W, A, D^{(tr)})$	$W^*(A) \approx W' = W - \eta_w \nabla_W L_{\text{tgt}}(W, A, D^{(tr)})$
II	The tester trains a data generator. $G^*(B) = \text{argmin}_G\ L_{dg}(G, B, D^{(tr)})$	$G^*(B) \approx G' = G - \eta_g \nabla_G L_{dg}(G, B, D^{(tr)})$
III	We apply the trained generator to generate a dataset. Then we train an auxiliary model on this dataset. $V^*(G^*(B)) = \text{argmin}_V\ L_{\text{tgt}}(V, f(G^*(B)))$	$V^*(G^*(B)) \approx V' = V - \eta_v \nabla_V L_{\text{tgt}}(V, f(G'))$
IV	We evaluate the learner on a validation set generated by the tester. The tester aims to maximize the validation loss. The learner aims to minimize this loss. $\min_A \max_B L_{\text{tgt}}(W^*(A), A, f(G^*(B))) - \lambda L_{\text{tgt}}(V^*(G^*(B)), D^{(\text{val})}) + \gamma L_{\text{tgt}}(W^*(A), A, D^{(\text{val})})$	$A \leftarrow A - \eta_a \nabla_A L_{\text{tgt}}(W', A, f(G')) + \gamma L_{\text{tgt}}(W', A, D^{(\text{val})})$ $B \leftarrow B + \eta_b \nabla_B (L_{\text{tgt}}(W', A, f(G')) - \lambda L_{\text{tgt}}(V', D^{(\text{val})}))$

2) On generated data in stage IV, we added a decorrelation regularizer [10], which accelerated convergence greatly and allowed us to reduce epoch number from 50 to 25 without compromising the quality of convergence.
3) We adopted parameter tying to make the representation learning layers of the auxiliary model and those of the discriminator in FQ-GAN [77] (which is used as the data generation model in our framework) share the same weight parameters. These parameters account for more than 95% of each model's total parameters. Tying these parameters greatly reduces parameter number, thereby decreasing computational costs of training them.
4) We reduced the pretraining time of the FQ-GAN to 4 h on either CIFAR-10 or CIFAR-100, by 1) using the methods proposed in [59], [60]; 2) reducing the number of iterations by half.
5) For pretraining the auxiliary model, the weight parameters of its feature extraction layers are set to those of the pretrained discriminator in FQ-GAN. We only need to pretrain the light-weight classification head (two feedforward layers) of the auxiliary model, which was efficiently finished in 20 min.

In addition, we provide an alternative way of generating adversarial validation examples, by automatically learning a data augmentation policy to augment data from original data and using augmented examples as adversarial validation data. The number of parameters in a data augmentation policy is much smaller than that in a deep generative model, and therefore is computationally more efficient to train. Following [42], we learn a differentiable augmentation policy P containing a set of subpolicies where each subpolicy consists of two operations and each operation has a probability and magnitude. The goal is to learn how to select subpolicies, which is formulated as a differentiable optimization problem using the Gumbel-Softmax [32] method. Given a training set $D^{(tr)}$, we randomly select a subset of $D^{(tr)}$. Then the learned P is applied to the selected subset to generate a set of augmented examples $f(P, D^{(tr)})$. The formulation in (9) becomes

$$\begin{aligned} \min_A \max_P \quad & L_{\text{tgt}}(W^*(A), A, f(P, D^{(tr)})) \\ & - \lambda L_{\text{tgt}}(V^*(P), D^{(\text{val})}) + \gamma L_{\text{tgt}}(W^*(A), A, D^{(\text{val})}) \\ s.t. \quad & V^*(P) = \operatorname*{argmin}_V\ L_{\text{tgt}}(V, f(P, D^{(tr)})) \\ & W^*(A) = \operatorname*{argmin}_W\ L_{\text{tgt}}(W, A, D^{(tr)}). \end{aligned} \tag{6}$$

IV. EXPERIMENTS

We apply our method for NAS in image classification. We use a conditional generative adversarial network (GAN) [17], [50]—FQ-GAN [77]—to generate validation examples. We randomly sample a class label y and a random noise vector z, then feed y and z into FQ-GAN to generate an image x. (x, y) is regarded as a generated image-label pair. We treat the momentum decay λ and FQ weight α in FQ-GAN as the learnable hyperparameters B in the data generation model. They are continuous scalars, which can be efficiently optimized using gradient-based methods by maximizing the loss in (4). We initialize them to the values reported in [77]. Please refer to Supplementary Section C.1 for more details regarding how these hyperparameters influence data generation. Following [45], we first perform architecture search, which finds an optimal cell, then perform architecture evaluation, which composes multiple copies of the searched cell into a large network, trains it from scratch, and evaluates it on a test set. Please refer to the supplementary material for detailed hyperparameter settings and additional results, such as computational costs, evaluation on adversarial examples, etc.

A. Datasets

We used five datasets: CIFAR-100 [38], CIFAR-10 [37], ImageNet [12], ImageNet-C [25], and CIFAR-10-C [25]. CIFAR-100 and CIFAR-10 contain 50K training images and 10K testing images, from 100 and 10 classes, respectively. For each dataset, we split the original 50K training set into a 25K new training set and a 25K validation set. ImageNet contains a training set of 1.3M images and a test set of 50K images, from 1000 object classes. Following [69], 10% of the 1.3M training images are randomly sampled to form a new training set, and another 2.5% of the 1.3M training images are randomly sampled to form a new architecture validation set. The ImageNet-C dataset consists of 15 diverse corruption types applied to validation images of ImageNet. The corruptions are drawn from four main categories: 1) noise; 2) blur; 3) weather; and 4) digital. CIFAR-10-C is a dataset generated by adding 15 common corruptions and four extra corruptions to CIFAR-10 test images.

Architectures are searched on CIFAR-100, CIFAR-10, and ImageNet, which are the most broadly used benchmark datasets in the NAS literature. To evaluate the worst-case performance of searched architectures, we manually select 2000 (20 per class) "worst-case" examples (whose class labels are considered by humans as being challenging to recognize) from the test set of CIFAR-100. In addition, we test these architectures on CIFAR-10-C and ImageNet-C.

B. Experimental Settings

Our framework is a general one that can be used together with any differentiable search method. Specifically, we apply our framework to the following NAS methods: DARTS [45], P-DARTS [7], PC-DARTS [69], and PR-DARTS [78]. The formulation in (9) is applied to DARTS and the formulation in (6) is applied to P-DARTS, PC-DARTS, and PR-DARTS.

During architecture evaluation, the combination of the training data and validation data is used to train a large network, stacking multiple copies of the searched cell. In addition to searching for architectures directly on ImageNet data, following [45], we also evaluate the architectures searched using CIFAR-10 and CIFAR-100 on ImageNet: given a cell searched using CIFAR-10 and CIFAR-100, multiple copies of it compose a large network, which is then trained on the 1.3M training data of ImageNet and evaluated on the 50K testing data.

The auxiliary model is set to ResNet-18 [23]. The tradeoff parameters λ and γ are tuned using a 5k held-out dataset in $\{0.1, 0.5, 1, 2, 3\}$. In most experiments, λ is set to 1 except for P-DARTS and PC-DARTS. For P-DARTS, λ is set to 0.5 for CIFAR-10 and 1 for CIFAR-100. For PC-DARTS, we use $\lambda = 3$ and $\lambda = 0.1$ for CIFAR-10 and CIFAR-100, respectively. γ is set to 1. Before running our method on a dataset D, we pretrain the data generation model (FQ-GAN) and auxiliary model (ResNet-18) on D. Note that our method did not unfairly use more data than baselines. The experiments were conducted on Nvidia 1080Ti GPUs.

We compare with the following curriculum learning (CL) methods: 1) self-paced CL (SPCL) [35]; and 2) dynamic instance hardness guided CL (DIHCL) [80]. These methods were originally designed for selecting training examples that have large training losses. We adapt them to select validation examples that have large validation losses, which are used to update architecture parameters.

TABLE II
CLASSIFICATION ERROR ON WORST-CASE, AVERAGE-CASE, AND ALL EXAMPLES IN CIFAR-100 TEST SET

Method	Worst-Case	Average-Case	All
Darts2nd	30.07 ± 0.36	18.82 ± 0.39	20.58 ± 0.44
SPCL-darts2nd	31.82 ± 0.12	17.60 ± 0.24	20.14 ± 0.27
DIHCL-darts2nd	30.88 ± 0.25	17.42 ± 0.33	19.86 ± 0.42
Ours-darts2nd (ours)	**27.01** ± 0.17	**16.08** ± 0.13	**18.55** ± 0.09
Pdarts	27.93 ± 0.36	15.82 ± 0.29	17.96 ± 0.15
SPCL-pdarts	28.08 ± 0.25	15.61 ± 0.11	17.71 ± 0.17
DIHCL-pdarts	28.36 ± 0.28	15.09 ± 0.26	18.05 ± 0.28
Ours-pdarts (ours)	**25.51** ± 0.16	**14.12** ± 0.15	**17.10** ± 0.13
Pcdarts	28.42 ± 0.15	14.95 ± 0.11	17.43 ± 0.13
SPCL-pcdarts	27.06 ± 0.28	14.26 ± 0.17	17.25 ± 0.09
DIHCL-pcdarts	27.94 ± 0.22	15.70 ± 0.19	17.69 ± 0.15
Ours-pcdarts (ours)	**24.92** ± 0.09	**13.36** ± 0.11	**16.17** ± 0.08
Prdarts	25.17 ± 0.14	14.77 ± 0.18	16.48 ± 0.06
SPCL-prdarts	25.33 ± 0.17	14.45 ± 0.14	16.76 ± 0.10
DIHCL-prdarts	25.80 ± 0.31	14.95 ± 0.26	16.83 ± 0.14
Ours-prdarts (ours)	**23.02** ± 0.11	**14.82** ± 0.10	**16.13** ± 0.02

Note: Bold indicate the best performance.

C. Results on "Worst-Case" Test Examples

1) Results on Worst-Case Examples in CIFAR-100 Test Set: Table II shows classification errors on 2000 "worst-case" examples selected from the CIFAR-100 test set. From this table, we can see that when our method is applied to different NAS baselines, including DARTS2nd, PC-DARTS, P-DARTS, and PR-DARTS, the classification errors of these baselines on worst-case test examples are significantly reduced (please refer to Supplementary Section B.5 for statistical significance test results). This demonstrates the effectiveness of our framework in improving the worst-case performance of NAS.

In our method, the learner improves its architecture's worst-case performance by minimizing the loss on the adversarial validation data generated by the tester. The adversarial validation sets are generated in a way that the learner's performance on these sets decreases. To verify this, we apply a ResNet-18 pretrained on the training set to make predictions on validation sets generated at different epochs. Fig. 2(left) shows that the errors consistently increase with epoch number. Fig. 3 shows some randomly sampled validation examples generated at different epochs. As can be seen, as the epoch increases, the images are more and more difficult to recognize.

We performed an automatic evaluation of images generated by our method Ours-darts2nd trained on CIFAR-100, using metrics including inception score [55] and Frechet inception distance (FID) [27]. Table III shows the results. Our method outperforms vanilla FQ-GAN and BigGAN. The reason is the images generated by our method are explicitly encouraged to

Fig. 2. (Left) Errors of a pretrained ResNet-18 on validation sets generated at different epochs. (Right) Randomly-sampled images from CIFAR-100 test set where ours-darts2nd makes correct predictions while vanilla darts-2nd makes incorrect predictions.

Fig. 3. Randomly sampled images from validation sets created at different epochs.

TABLE III
AUTOMATIC EVALUATION OF GENERATED CIFAR-100 IMAGES

Method	Inception↑	FID↓
BigGAN	9.36 ± 0.10	9.01 ± 0.44
FQ-GAN	9.59 ± 0.04	7.42 ± 0.07
Ours	$\mathbf{9.75} \pm 0.03$	$\mathbf{6.01} \pm 0.09$

Note: Bold indicate the best performance.

be "meaningful". They are used to train an auxiliary model, which is then evaluated on a human-labeled validation set. If generated images have poor quality, the auxiliary model trained by them will perform poorly on the human-labeled validation set. Our framework prevents this from happening by explicitly minimizing the loss on the human-labeled validation set. Such a mechanism is lacking in vanilla FQ-GAN and BigGAN. Fig. 3 shows some randomly sampled images generated by our method. As can be seen, these images are realistic.

These generative adversarial validation sets can help the learner to identify the worst-case weakness of its architecture and provide guidance on how to improve it. Different from baseline NAS methods which optimize architectures by minimizing an average-case validation loss calculated on a single fixed validation set, our method improves its architecture by minimizing the loss on each generated adversarial validation set. By doing this, architectures searched by our method can make more accurate predictions on worst-case examples during test time.

In this table, we also see that our method works better than the two curriculum learning (CL) methods: 1) SPCL; and 2) DIHCL. In our method, adversarial validation data generation and architecture search are performed jointly in an end-to-end framework, whereas in the two baselines, validation example selection is performed separately from architecture search. Performing the two tasks end-to-end enables the generation of adversarial validation examples to be guided dynamically by the quality of the learner's architectures. In contrast, SPCL and DIHCL identify hard examples based on high observed losses within a fixed dataset and lack the ability to explore outside this empirical distribution. Our method, by leveraging a deep generative model, can synthesize new and diverse inputs that probe the learner beyond the support of the original data. This enables broader discovery of underrepresented failure modes. Moreover, while SPCL and DIHCL use heuristic example selection strategies, our generator is optimized adversarially to expose the learner's weaknesses in a targeted way. These differences make our approach fundamentally more expressive and adaptive, leading to superior worst-case robustness.

In Table II, we also report performance on all test images and the 8000 "average-case" images. As can be seen, our method's performance on average-case test data is on par with that of baselines. This demonstrates that our framework is able to improve NAS' worst-case performance without sacrificing average-case performance. The reason is that in (4), the architecture is updated by simultaneously minimizing the worse-case validation loss (the first term) and the average-case validation loss (the second term). By seeking a proper tradeoff between these two losses, our framework is able to maintain average-case performance while significantly boosting worst-case performance.

TABLE IV
AVERAGE CLASSIFICATION ERROR (%) ON CIFAR-10-C

Method	Error
Darts2nd	23.8 ± 0.7
SPCL-Darts2nd	23.5 ± 0.9
DIHCL-Darts2nd	22.9 ± 1.2
Ours-Darts2nd	**20.1** ± 0.4
Pdarts	22.4 ± 1.1
SPCL-Pdarts	22.7 ± 0.7
DIHCL-Pdarts	22.1 ± 1.0
Ours-Pdarts	**18.9** ± 0.9
Pcdarts	23.0 ± 0.6
SPCL-Pcdarts	22.8 ± 0.4
DIHCL-Pcdarts	22.5 ± 0.9
Ours-Pcdarts	**19.8** ± 0.5
Prdarts	21.5 ± 0.9
SPCL-Prdarts	21.2 ± 0.6
DIHCL-Prdarts	21.8 ± 1.0
Ours-Prdarts	**19.1** ± 0.3

Note: Architectures are searched on CIFAR-10. Bold indicate the best performance.

TABLE V
MEAN CORRUPTION ERROR (MCE, %) ON IMAGENET-C

Method	mCE
Pcdarts	78.8
SPCL-Pcdarts	78.4
DIHCL-Pcdarts	79.1
Ours-Pcdarts	**75.4**

Note: Architectures are searched on ImageNet. Bold indicate the best performance.

2) Results on CIFAR-10-C and ImageNet-C: We evaluate different methods on CIFAR-10-C and ImageNet-C [25]. For CIFAR-10-C experiments, the architectures were searched on CIFAR-10. For ImageNet-C experiments, the architectures were searched on ImageNet and our framework was applied to PCDARTS (we did not experiment with other NAS baselines such as DARTS which are computationally too costly to perform NAS on ImageNet). Tables IV and V show the average classification error on CIFAR-10-C and the mean Corruption Error on ImageNet-C [25], respectively. Lower is better. Our methods outperform baselines, which demonstrates that our methods are more robust than baselines against data corruptions in CIFAR-10-C and ImageNet-C. The reason is: examples in CIFAR-10-C and ImageNet-C can be considered as a type of worst-case examples; our method measures and robustifies models' worst-case performance by using deep generative models to generate worst-case examples, and therefore has better capability to handle examples in CIFAR-10-C and ImageNet-C.

TABLE VI
TEST ERRORS ON CIFAR-10 AND CIFAR-100 IN THE EVALUATION OF ROBUSTNESS AGAINST PERFORMANCE COLLAPSE

Data	Space	DARTS	RDARTS-L2	DARTS-ES	DARTS-	SDART	SPCL	Ours
CIFAR-10	S1†	4.69	3.46	3.93	3.34	3.26	4.02	**3.08**
	S1‡	3.84	2.78	3.01	2.68	2.73	3.29	**2.55**
	S2	5.54	3.31	4.07	4.03	3.11	4.50	**2.91**
	S3	3.92	2.51	3.55	2.95	3.07	4.26	**2.46**
	S4	8.33	3.56	4.69	4.14	3.49	5.72	**3.28**
CIFAR-100	S1	29.46	24.25	28.37	22.41	22.33	26.72	**21.38**
	S2	26.05	22.24	23.25	21.61	20.56	24.10	**19.45**
	S3	28.90	23.99	23.73	21.13	21.08	20.79	**19.92**
	S4	22.85	21.94	21.26	21.55	21.25	23.38	**20.33**

Note: † denotes that the initial channel number is 16 and cell number is 8. ‡ denotes that the initial channel number is 36 and cell number is 20. We compared with DARTS [45], RDARTS-L2 [75], DARTS-ES [75], DARTS- [8], SDART [5], and SPCL [35]. Bold indicate the best performance.

D. Robustness Against Performance Collapse

Next, we present another reason for our method's effectiveness. Our method optimizes the architecture by minimizing the loss on each generated adversarial validation set. As a result, the learned architecture is more robust against performance collapse. Several studies [5], [8], [75] have shown that differentiable NAS methods are prone to performance collapse. To empirically show the robustness of our method, we evaluate it on four search spaces designed for measuring robustness in [75]. Following RobustDARTS-L2 [75], we set cell number to 8 and initial channel number to 16 for both CIFAR-10 and CIFAR-100. The same as the protocols in DARTS [45] and RobustDARTS [75], architecture search runs for four times with random initialization.

A searched architecture is retrained from scratch for a few epochs. The architecture achieving the highest validation accuracy is selected. Architecture evaluation is performed on selected architectures. Table VI shows the results. As can be seen, on the four spaces, our method outperforms baseline methods. This demonstrates that our method is more robust against performance collapse.

Compared with SDARTS, which improves robustness to optimization noise and overfitting via perturbation-based regularization, our method directly targets robustness in the data space by adversarially generating diverse, semantically valid, and informative examples. This approach leads to more effective identification and resolution of model vulnerabilities.

E. Overall Errors on CIFAR-10 and ImageNet

In this section, we evaluate architectures' overall performance on entire test sets, which include both difficult examples and average-case examples.

1) Results on CIFAR-10: Table VII shows the classification error (%), number of weight parameters (millions), and search cost (GPU days) of different NAS methods on the entire test set of CIFAR-10. When our framework is applied to NAS baselines, the overall performance of these baselines is significantly improved. This is because our framework can improve worst-case performance without sacrificing average-case performance, as shown in Table II, which results in improved overall performance.

TABLE VII
RESULTS ON CIFAR-10, INCLUDING CLASSIFICATION ERROR (%) ON THE TEST SET, NUMBER OF PARAMETERS (MILLIONS) IN THE SEARCHED ARCHITECTURE, AND SEARCH COST (GPU DAYS)

Method	Error	Param.	Cost
*ResNet [23]	6.43	1.7	-
*DenseNet [31]	3.46	25.6	-
*PNAS [43]	3.41 ± 0.09	3.2	150
*ENAS [52]	2.89	4.6	0.5
*AmoebaNet [54]	2.55 ± 0.05	3.1	3150
*GDAS [13]	2.93	3.4	0.2
*R-DARTS [75]	2.95 ± 0.21	-	1.6
*DARTS+PT [66]	2.61 ± 0.08	3.0	0.8
*DARTS$^-$ [8]	2.59 ± 0.08	3.3	0.4
*DropNAS [28]	2.58 ± 0.14	4.4	0.7
*DrNAS [6]	2.54 ± 0.03	4.0	0.4
*ISTA-NAS [72]	2.54 ± 0.05	3.3	0.1
*MiLeNAS [22]	2.51 ± 0.11	3.9	0.3
*GAEA [41]	2.50 ± 0.06	-	0.1
*PDARTS-ADV [5]	2.48 ± 0.02	3.4	1.1
*DOTS [21]	2.49 ± 0.06	4.1	0.3
*β-DARTS [73]	2.53 ± 0.08	3.8	0.4
*AGNAS [63]	$2.53 \pm .003$	3.6	0.4
*RF-DARTS [76]	2.60	4.6	-
*GENAS [70]	2.49 ± 0.03	3.2	0.3
*Darts2nd [45]	2.76 ± 0.09	3.1	4.0
SPCL-darts2nd [35]	2.81 ± 0.23	3.2	5.1
DIHCL-darts2nd [80]	2.86 ± 0.10	3.4	5.4
Ours-darts2nd (ours)	**2.68** ± 0.03	3.1	4.0
†Pcdarts [69]	2.57 ± 0.07	3.9	0.1
SPCL-pcdarts [35]	2.69 ± 0.14	3.9	0.2
DIHCL-pcdarts [80]	2.73 ± 0.05	4.0	0.4
Ours-pcdarts (ours)	**2.50** ± 0.05	3.7	0.1
*Pdarts [7]	2.54 ± 0.04	3.6	0.3
SPCL-pdarts [35]	2.77 ± 0.25	3.8	0.6
DIHCL-pdarts [80]	2.85 ± 0.12	3.8	0.9
Ours-pdarts (ours)	**2.48** ± 0.04	3.6	0.3
†Prdarts [78]	2.37 ± 0.03	3.4	0.2
SPCL-prdarts [35]	2.44 ± 0.05	3.3	0.3
DIHCL-prdarts [80]	2.48 ± 0.06	3.5	0.5
Ours-prdarts (ours)	**2.31** ± 0.02	3.3	0.2

Note: Ours-darts2nd denotes that our method is applied to the search space of DARTS. Similar meanings hold for other notations in such a format. * means the results are taken from DARTS$^-$ [8], β-DARTS [73], and AGNAS [63]. † means we reran this method for 10 times. Search cost is measured by GPU days. Bold indicate the best performance.

TABLE VIII
RESULTS ON IMAGENET, INCLUDING TOP-1 AND TOP-5 CLASSIFICATION ERRORS ON THE TEST SET

Method	Top-1	Top-5
*Inception-v1 [64]	30.2	10.1
*MobileNet [29]	29.4	10.5
*ShuffleNet 2× (v2) [47]	25.1	7.6
*NASNet-A [83]	26.0	8.4
*AmoebaNet-C [54]	24.3	7.6
*SNAS-CIFAR10 [68]	27.3	9.2
*DSNAS-ImageNet [30]	25.7	8.1
*PCDARTS-CIFAR10 [69]	25.1	7.8
*ProxylessNAS-ImageNet [4]	24.9	7.5
*FairDARTS-ImageNet [9]	24.4	7.4
*DrNAS-ImageNet [6]	24.2	7.3
*PR-DARTS [78]	24.1	7.3
*DARTS$^-$-ImageNet [8]	23.8	7.0
*DOTS [21]	24.3	7.4
*β-DARTS [73]	23.9	7.0
*RF-PCDARTS [76]	23.9	7.1
*GENAS [70]	23.9	7.2
*Darts2nd-cifar10 [45]	26.7	8.7
SPCL-darts2nd-cifar10 [35]	26.4	8.5
DIHCL-darts2nd-cifar10 [80]	26.8	8.9
Ours-darts2nd-cifar10 (ours)	**25.8**	**8.2**
*Pdarts-cifar10 [7]	24.4	7.4
SPCL-pdarts-cifar10 [35]	24.4	7.4
DIHCL-pdarts-cifar10 [80]	24.6	7.5
Ours-pdarts-cifar10 (ours)	**24.1**	**7.1**
*Pdarts-cifar100 [7]	24.7	7.5
SPCL-pdarts-cifar100 [35]	24.5	7.4
DIHCL-pdarts-cifar100 [80]	24.7	7.6
Ours-pdarts-cifar100 (ours)	**24.0**	**7.1**
*Pcdarts-imagenet [69]	24.2	7.3
SPCL-pcdarts-imagenet [35]	24.0	7.2
DIHCL-pcdarts-imagenet [80]	24.1	7.2
Ours-pcdarts-imagenet (ours)	**23.5**	**6.7**

Note: * means the results are taken from DARTS$^-$ [8], DrNAS [6], and β-DARTS [73]. The rest of the notations are the same as those in Table VII. Bold indicate the best performance.

2) Results on ImageNet: Table VIII shows results on ImageNet. Ours-pcdarts-imagenet where the architecture is searched on ImageNet performs much better than Pcdarts-imagenet and achieves the lowest errors among all methods in Table VIII. Ours-pdarts-cifar100, Ours-pdarts-cifar10, and Ours-darts2nd-cifar10, where the architectures are searched on CIFAR-10 or CIFAR-100 and evaluated on ImageNet, outperform their corresponding baselines Pdarts-cifar100, Pdarts-cifar10, and Darts2nd-cifar10. These results further demonstrate the effectiveness of our method in improving NAS' overall performance.

F. Results on ImageNet-A, ImageNet-R, and ImageNet-Sketch

We also evaluated our method on ImageNet-A [26], ImageNet-R [24], and ImageNet-Sketch [67]. The experimental settings are the same as those described in Section IV-B. Our framework was applied to PCDARTS. Tables IX–XI show the results. As can be seen, our method outperforms baselines, which further demonstrates the effectiveness of our method in searching for worst-case robust neural architectures.

TABLE IX
ACCURACY ON IMAGENET-A

Method	Accuracy
Pcdarts	6.2
SPCL-Pcdarts	6.6
DIHCL-Pcdarts	6.3
Ours-Pcdarts	**10.1**

Note: Architectures are searched on ImageNet. Bold indicate the best performance.

TABLE X
TOP-1 ERROR ON IMAGENET-R

Method	Accuracy
Pcdarts	62.3
SPCL-Pcdarts	62.7
DIHCL-Pcdarts	61.9
Ours-Pcdarts	**59.5**

Note: Architectures are searched on ImageNet. Bold indicate the best performance.

TABLE XI
TOP-1 ACCURACY ON IMAGENET-SKETCH

Method	Accuracy
Pcdarts	12.8
SPCL-Pcdarts	12.9
DIHCL-Pcdarts	12.5
Ours-Pcdarts	**16.1**

Note: Architectures are searched on ImageNet. Bold indicate the best performance.

G. Evaluation on Medical Imaging Tasks

We performed additional experiments on two clinical applications. The first task involves brain tumor classification from magnetic resonance imaging (MRI) scans. We used the publicly available dataset from [1], which contains 3,264 images across four categories: 1) Glioma; 2) Meningioma; 3) Pituitary; and 4) Healthy. The dataset was divided into a training set of 2,870 images and a test set of 394 images. From the test set, a board-certified physician identified 79 cases that are particularly challenging for tumor classification, such as those with overlapping boundaries or heterogeneous tumor textures. The second task focuses on pneumonia diagnosis from chest X-rays. We used the dataset provided by [36], which includes 5,863 images from two categories: 1) pneumonia; and 2) normal. The dataset was split into 4,690 training images and 1,173 test images. Among the test samples, 152 cases were identified by a clinical expert as difficult to diagnose, typically due to subtle or diffuse opacities. In both tasks, our method was applied to DARTS-2nd. The training data were further divided into new training and validation subsets with a ratio of 1:1, corresponding to $D^{(tr)}$ and $D^{(\mathrm{val})}$ in our formulation. The hyperparameters λ and γ were set to 1, and all other settings followed Section IV-B. The baseline methods were trained on the entire training dataset, including both $D^{(tr)}$ and $D^{(\mathrm{val})}$.

TABLE XII
AUC ON TWO MEDICAL IMAGING DIAGNOSIS TASKS

Method	Brain Tumor	Pneumonia
Darts2nd	0.68 ± 0.06	0.73 ± 0.03
SPCL-darts2nd	0.61 ± 0.04	0.71 ± 0.04
DIHCL-darts2nd	0.63 ± 0.07	0.67 ± 0.06
Ours-darts2nd	$\mathbf{0.77} \pm 0.03$	$\mathbf{0.84} \pm 0.02$

Note: Bold indicate the best performance.

TABLE XIII
ERRORS FOR "ADVERSARIAL ONLY" ON TEST SETS OF CIFAR-100 (C100) AND CIFAR-10 (C10)

Method	Error (%)
Adversarial only (Darts2nd, C100)	20.52 ± 0.11
Ours (Darts2nd, C100)	$\mathbf{18.55} \pm 0.09$
Adversarial only (Pdarts, C100)	18.45 ± 0.16
Ours (Pdarts, C100)	$\mathbf{16.17} \pm 0.08$
Adversarial only (Darts2nd, C10)	2.84 ± 0.06
Ours (Darts2nd, C10)	$\mathbf{2.68} \pm 0.03$

Note: Bold indicate the best performance.

Table XII reports the area under ROC curve (AUC) on the challenging subsets from both datasets. Our method achieves substantially higher error rates than all baselines, demonstrating that the proposed MLO framework enhances robustness on real-world, safety-critical medical imaging tasks.

H. Ablation Studies

To investigate the importance of encouraging the generated validation examples to be meaningful, we perform an ablation study of "adversarial only": the tester generates validation examples solely by maximizing the learner's validation loss, without considering their meaningfulness. Accordingly, the third stage in our framework where the tester trains an auxiliary model on generated validation examples is removed and λ at the fourth stage is set to 0. For CIFAR-100, our method is applied to P-DARTS and DARTS-2nd. For CIFAR-10, our method is applied to DARTS-2nd. Table XIII shows the results. On both CIFAR-10 and CIFAR-100, it is more advantageous to generate validation examples that are both meaningful and result in the degradation of the learner's performance, rather than focusing solely on impairing the learner's performance with the generated validation examples. The reason is that without being meaningful, the generated validation examples could be outliers that hurt NAS performance.

Fig. 4 shows some images randomly sampled from validation examples generated under "adversarial only". As can be seen, these images are difficult to recognize even for humans or contain labeling errors. Even a highly accurate model cannot

Fig. 4. Randomly sampled images that are generated under "adversarial-only" and under "adversarial+meaningful".

TABLE XIV
ERRORS ON CIFAR-100 TEST SET, UNDER DIFFERENT TESTER NETWORKS

Method	Error (%)
SNGAN-darts2nd	19.82 ± 0.11
BigGAN-darts2nd	19.69 ± 0.18
FQGAN-darts2nd	**18.55** ± 0.09
SNGAN-pcdarts	17.84 ± 0.09
BigGAN-pcdarts	17.91 ± 0.07
FQGAN-pcdarts	**17.10** ± 0.13
SNGAN-pdarts	17.22 ± 0.09
BigGAN-pdarts	17.03 ± 0.12
FQGAN-pdarts	**16.17** ± 0.08

Note: Bold indicate the best performance.

achieve good performance on such erratic examples. In contrast, validation images generated by "adversarial + meaningful" are more semantically meaningful. Our method promotes meaningfulness of generated examples by encouraging these examples to be useful for training a high-performance auxiliary model. The results demonstrate that this is an effective way of improving meaningfulness.

We perform another ablation study on how the data generation power of testers affects the learner's performance. We experimented with three testers: 1) SN-GAN [51]; 2) BigGAN [3]; and 3) FQ-GAN [77]. As reported in [77], FQ-GAN performs better than BigGAN and SN-GAN in generating high-fidelity images. Table XIV shows that using FQ-GAN yields better NAS performance than using BigGAN and SN-GAN. The reason is that with a more powerful data generator, the tester can more effectively generate adversarial and meaningful validation examples. Validation examples with better quality can evaluate the learner more effectively and help to improve the learner's architecture.

For the generative model of the tester, we perform the following ablation studies. The studies are performed on Ours-darts. The datasets are Cifar-100 and Cifar-10. The generative model is FQ-GAN [77].

1) *Weights-Only:* We fix the meta parameters of FQ-GAN and only learn its weight parameters. The meta parameters of FQ-GAN are set to the default values in [77]. For the generator G in FQ-GAN, we learn it at the second stage. For the discriminator S, we learn it at the fourth stage. Let L_{fqgan} denote the FQ-GAN loss. The corresponding formulation is

$$\begin{aligned}\min_{A}\max_{S}\quad & L_{\text{tgt}}(W^*(A), f(G^*(S), z_2)) \\ & - \lambda L_{\text{tgt}}(V^*(G^*(S)), D^{(\text{val})}) \\ s.t.\quad & V^*(G^*(S)) = \operatorname*{argmin}_{V} L_{\text{tgt}}(V, f(G^*(S), z_1)) \\ & G^*(S) = \operatorname*{argmin}_{G} L_{\text{fqgan}}(G, S, D^{(tr)}) \\ & W^*(A) = \operatorname*{argmin}_{W} L_{\text{tgt}}(A, W, D^{(tr)}). \end{aligned} \tag{7}$$

2) *Pretrain:* We use a fixed-weights FQ-GAN pretrained by [77] instead of learning it in our framework. We use the pretrained FQ-GAN to generate $M = 100K$ image-label pairs $\{(p_i, q_i)\}_{i=1}^{M}$ where q_i is a label and p_i is the corresponding image, then select an adversarial validation set from the 100K pairs. For each generated pair, we associate it with a selection variable $s \in [0, 1]$. The larger s is, the more likely that the pair is selected. Let $\ell_{\text{tgt}}(W^*(A), p_i, q_i)$ denote the loss defined on the pair (p_i, q_i). The corresponding formulation is

$$\begin{aligned}\min_{A}\max_{\{s_i\}_{i=1}^{M}}\quad & \frac{1}{\sum_{i=1}^{M} s_i}\sum_{i=1}^{M} s_i \ell_{\text{tgt}}(W^*(A), p_i, q_i) \\ & + \lambda L_{\text{tgt}}(W^*(A), D^{(\text{val})}) \\ s.t.\quad & W^*(A) = \operatorname*{argmin}_{W} L_{\text{tgt}}(A, W, D^{(tr)}). \end{aligned} \tag{8}$$

At the second stage, we learn the selection variables $\{s_i\}_{i=1}^{M}$ by maximizing the learner's validation loss $\sum_{i=1}^{M} s_i \ell_{\text{tgt}}(W^*(A), p_i, q_i)$. We learn the architecture A by minimizing the loss $\sum_{i=1}^{M} s_i \ell_{\text{tgt}}(W^*(A), p_i, q_i)$ on selected validation examples and minimizing the loss $L_{\text{tgt}}(W^*(A), D^{(\text{val})})$ on the human-provided validation set $D^{(\text{val})}$. λ is a trade-off parameter. $(1/\sum_{i=1}^{M} s_i)$ is a normalization term.

In our full method, both the meta parameters and weight parameters of FQ-GAN are learned. Table XV shows the results. We make two observations. First, our full method works better than Weights-only. The reason is in weights-only, the values of meta parameters are fixed, and these values may not be optimal for generating adversarial validation sets. In contrast, our full method learns these meta parameters by maximizing the validation loss and the meta parameters are optimized specifically for

TABLE XV
TEST ERRORS ON CIFAR-100 AND CIFAR-10

Method	Cifar-100	Cifar-10
Weights-only	19.42 ± 0.21	2.71 ± 0.02
Pretrain	19.77 ± 0.15	2.74 ± 0.03
Our full method	**18.55** ± 0.09	**2.68** ± 0.03

Note: Bold indicate the best performance.

TABLE XVI
TEST ERRORS ON CIFAR-100 AND CIFAR-10

Method	Cifar-100	Cifar-10
BLO	20.06 ± 0.24	2.74 ± 0.08
MLO	**18.55** ± 0.09	**2.68** ± 0.03

Note: Bold indicate the best performance.

achieving the goal of generating high-fidelity adversarial validation sets. Second, our full method and weights-only work better than pretrain. The reason is in pretrain, weights parameters are fixed and they may not be optimal for generating adversarial validation sets. In our full method and weights-only, the weights parameters are learned specifically for achieving the goal of generating high-fidelity adversarial validation sets.

In addition, we conducted an ablation study to assess whether a simpler bilevel optimization (BLO) alternative could achieve comparable performance. The BLO formulation is given as follows:

$$\begin{aligned} \min_{A} \max_{B} \quad & L_{\text{tgt}}(W^*(A), A, f(G^*(B))) \\ & - \lambda L_{\text{tgt}}(V^*(B), D^{(\text{val})}) \\ & + \gamma L_{\text{tgt}}(W^*(A), A, D^{(\text{val})}) \\ s.t. \quad & V^*(B), G^*(B), W^*(A) \\ & = \underset{V,G,W}{\text{argmin}} \; L_{\text{tgt}}(V, f(G)) \\ & + L_{\text{dg}}(G, B, D^{(tr)}) + L_{\text{tgt}}(W, A, D^{(tr)}). \end{aligned} \tag{9}$$

Table XVI compares this BLO variant with our original MLO formulation on CIFAR-100 and CIFAR-10, both applied to DARTS-2nd. As shown, MLO achieves consistently lower test errors, demonstrating its superiority in guiding architecture discovery. The performance difference arises because the BLO formulation collapses multiple coupled optimization processes—learner, tester, and auxiliary evaluator—into a single lower-level problem, thereby losing the hierarchical dependency structure that governs their interactions. In the BLO setup, gradients from the tester and auxiliary model are aggregated without explicit separation of their objectives, which limits the learner's ability to balance adversarial difficulty and semantic meaningfulness of generated data. In contrast, the MLO formulation explicitly isolates these dependencies across distinct optimization layers. This structured coordination allows MLO to approximate a worst-case yet meaningful validation landscape, leading to architectures that generalize more robustly.

TABLE XVII
SEARCH COST (SC) IN GPU HOURS AND MEMORY COSTS (MC) IN MIB OF DIFFERENT METHODS

Method	Error-C100	Error-C10	Param.	SC	MC
Darts2nd	20.58 ± 0.44	2.76 ± 0.09	3.1	4.0	11053
NCR-darts2nd (ours)	18.42 ± 0.11	2.67 ± 0.03	3.3	6.1	19226
CR-darts2nd (ours)	18.55 ± 0.09	2.68 ± 0.03	3.1	3.9	10982
Pcdarts	17.96 ± 0.15	2.57 ± 0.07	3.9	0.1	10058
NCR-pcdarts (ours)	17.01 ± 0.18	2.50 ± 0.07	4.0	0.7	18429
CR-pcdarts (ours)	17.10 ± 0.13	2.50 ± 0.05	3.7	0.1	10071
Pdarts	17.43 ± 0.13	2.54 ± 0.04	3.6	0.3	9659
NCR-pdarts (ours)	16.14 ± 0.06	2.48 ± 0.05	3.8	1.1	18581
CR-pdarts (ours)	16.17 ± 0.08	2.48 ± 0.04	3.6	0.3	9597
Prdarts	16.48 ± 0.06	2.37 ± 0.03	3.4	0.2	10159
NCR-prdarts (ours)	16.14 ± 0.02	2.31 ± 0.01	3.4	0.8	20188
CR-prdarts (ours)	16.13 ± 0.02	2.31 ± 0.02	3.3	0.2	10195

In Supplementary Section B.1, we experimented with additional ablation studies: 1) sensitivity analysis of λ and γ; and 2) training the generator from scratch. Our full method outperforms these ablation settings, which further demonstrates the effectiveness of the individual components in our method.

I. Search Cost and Memory Cost

Table XVII shows search costs (GPU hours) and memory costs (MiB) of different methods. CR and NCR denote our method before and after applying the search cost reduction methods described in Section III-C, respectively. As can be seen, the search costs of CR are no more than those of baselines, while the classification errors of CR are significantly lower than those of baselines. CR has lower costs than NCR while achieving classification performance similar to that of NCR. These observations demonstrate the effectiveness of these cost reduction methods. We applied some of these cost-saving strategies on baselines such as DARTS, including decreasing the frequency of architecture updates from every iteration to every eighth iteration, and reducing the number of epochs from 50 to 25. However, these modifications resulted in a significant decline in performance. In light of that, we retained the default hyperparameters for the NAS baselines. Other cost-reduction methods, such as parameter tying and minimizing the pretraining time of the data generation model, cannot be applied to the NAS baselines.

V. DISCUSSION

Our work introduces a new type of adversarial data—generative adversarial validation examples (GAVEs)—that differs fundamentally from traditional adversarial examples. Traditional adversarial examples are created by applying small, often imperceptible perturbations to real data points to induce misclassification, and they primarily evaluate local robustness around existing examples. In contrast, GAVEs are synthesized entirely from scratch by a deep generative model, without relying on real input instances. They are not constrained to resemble any specific training examples and can diverge significantly in both appearance and semantics. This allows GAVEs to probe global weaknesses of the model, evaluating its performance over a broad and potentially underrepresented region of the input distribution. While traditional adversarial examples target

robustness to fine-grained perturbations, GAVEs are designed to assess and improve worst-case generalization. As shown in Supplementary Section B.2, although our framework is not optimized for defending against traditional adversarial attacks, it still exhibits improved performance under such perturbations. These results highlight the complementary role of GAVEs in characterizing failure modes that go beyond local robustness, providing a broader lens for evaluating and enhancing the reliability of neural architectures.

As our method relies on a deep generative model to synthesize adversarial validation examples, the quality of the generated data plays a pivotal role in guiding NAS. A key challenge is that poorly generated examples—such as outliers or mislabeled instances—can mislead the optimization process and degrade the resulting architecture's performance. To mitigate this, our framework incorporates a meaningfulness constraint that encourages the generated examples to be not only adversarial (i.e., capable of revealing model weaknesses) but also useful for learning. Specifically, the generative model is trained to produce examples that both decrease the learner's performance and enable an auxiliary model trained on these examples to perform well on a human-labeled validation set. This dual objective, formalized in (4), discourages the generation of low-fidelity or semantically irrelevant data. Our ablation studies in Section IV-G support this design: removing the meaningfulness constraint ("adversarial only") or using a fixed, nonadaptive generator ("pretrain") leads to a noticeable drop in NAS performance. These findings underscore the importance of maintaining high-quality generative modeling and highlight our framework's robustness to potential failures in the data generation process.

Since our framework relies on generating adversarial validation datasets, it is important to understand how the distribution of the generated data differs from the original training distribution. The generative model is adversarially optimized to produce examples that degrade the learner's performance, intentionally shifting the data distribution toward underrepresented or high-loss regions. At the same time, the auxiliary model enforces a meaningfulness constraint, ensuring that the generated examples remain semantically valid and visually coherent. To quantify this distributional difference, we computed the Fréchet inception distance (FID) between the generated adversarial validation examples and the original training data. As reported in Table III, the FID score is 6.01, indicating that the generated samples deviate moderately from the training data—as expected given their adversarial purpose—while maintaining distributional realism. These results support the interpretation that the generator targets informative edge cases rather than producing outliers.

Our framework is designed to generate a diverse set of adversarial validation examples that expose different types of model weaknesses, rather than concentrating on narrow or repetitive failure modes. This is achieved through two mechanisms. First, the generative model includes learnable hyperparameters that are optimized adversarially to degrade the learner's performance. Unlike handcrafted perturbations, this optimization is unconstrained by predefined attack patterns and encourages the generator to explore a broad space of challenging inputs. Second, to prevent the generator from collapsing onto a limited set of hard but uninformative examples, we incorporate a meaningfulness constraint: an auxiliary model trained on the generated data must generalize well to a human-labeled validation set. This encourages the generated examples to be semantically rich and broadly useful. As demonstrated in our ablation study (Table XIII), omitting this constraint leads to degenerate or incoherent samples, while our full method produces more diverse and plausible examples that reflect a range of difficult scenarios.

Although our experiments are conducted on standard computer vision benchmarks, the proposed framework has clear relevance to safety-critical domains such as healthcare. In clinical settings, architectures that generalize well on average may still fail on rare conditions, leading to high-risk consequences. Our method addresses this concern by optimizing architectures to perform well on challenging, adversarially generated validation examples that reflect potential worst-case scenarios. For example, generative models could be trained to synthesize underrepresented pathological patterns, enabling NAS to search for architectures that remain robust on diagnostically difficult edge cases. This makes our approach well-suited for deployment in domains where worst-case performance, not just average accuracy, is a key requirement.

While our four-stage nested optimization framework demonstrates strong empirical performance, we acknowledge the lack of formal theoretical guarantees regarding its convergence and stability. Recent works on bilevel optimization have provided convergence analyses under specific smoothness and regularity conditions [11], [33]. These studies employ techniques such as implicit differentiation, truncated backpropagation, and hierarchical update schemes to analyze the interaction between optimization levels. Although our method involves more than two levels, we believe these analytical tools can be extended to multilevel settings by recursively applying similar principles to each nested layer. A rigorous theoretical treatment of our framework is an important direction for future work. In the meantime, our extensive empirical results across multiple datasets and search spaces, along with ablation studies and stability evaluations (Section A.3 in the supplements), suggest that the optimization process is stable and effective in practice.

While our work is primarily empirical, the effectiveness of minimizing worst-case validation loss can be understood through theoretical intuitions drawn from robust learning. Minimizing worst-case loss encourages robustness to underrepresented or high-loss regions of the data distribution, which helps reduce sensitivity to distributional shifts—particularly valuable in settings where test-time data may differ from training data in sparse but systematic ways. This aligns with recent theoretical frameworks [14], [58] in robust generalization and adversarial training, which suggest that minimizing the maximal loss over a rich set of inputs can improve generalization to unseen or atypical examples. These works also formalize the robustness–overfitting tradeoff, showing that incorporating worst-case constraints can reduce variance in generalization

error across subpopulations. Although a full theoretical analysis of our MLO procedure is beyond the scope of this work, we believe our method benefits from similar principles by optimizing architectures to perform well on adversarially generated, high-loss validation examples.

As detailed in Section A of the Supplements, our optimization algorithm is built upon the well-established framework of [45], whose theoretical convergence properties have been analyzed in prior studies [16], [20], [33], [46], [71]. Our formulation inherits these convergence properties because it follows the same one-step gradient-based approximation and finite-difference scheme used in differentiable NAS. Moreover, we explicitly demonstrate stable convergence behavior under different optimization schedules. As shown in Figs. 4–6 of the Supplements, the objective functions of all optimization variables converge smoothly and monotonically under diverse training configurations—including varying batch sizes (64 versus 128) and learning rates (0.025 versus 0.05). Across these distinct schedules, the loss trajectories consistently decrease, indicating that the proposed MLO remains stable regardless of hyperparameter scaling or update frequency. This empirical ablation demonstrates that convergence is robust to changes in optimization dynamics. Second, we empirically verify that the final architectures improve progressively during training (Fig. 7 in the Supplements), further demonstrating convergence in architecture space. Third, regarding the GAN-based tester, we mitigate potential generator quality issues through two complementary mechanisms.

1) An auxiliary classifier V is jointly optimized to assess the meaningfulness of generated examples. This classifier provides an explicit feedback signal—if generated data fail to improve V's validation performance, the generator's updates are automatically suppressed. This design encourages that only semantically valid and informative samples contribute to architecture updates, preventing low-fidelity or mode-collapsed examples from misleading the search.
2) In addition, the generator is initialized from a pretrained FQ-GAN, which provides realistic and diverse samples at the early stage, further stabilizing training.

A key limitation of our framework is that it is currently restricted to differentiable NAS methods, where architecture parameters can be updated via gradient-based optimization. This differentiability is crucial for enabling efficient end-to-end optimization with respect to adversarially generated validation examples. Consequently, our method is not directly compatible with nondifferentiable NAS approaches, such as those based on reinforcement learning or evolutionary algorithms, which rely on discrete search and lack gradient information. Nonetheless, we view our method as a foundational step that could be extended in future work. One promising direction is to decouple the generative adversarial validation mechanism from architecture optimization, allowing it to serve as a module that refines fitness evaluations or selection strategies in nondifferentiable settings. Such extensions could broaden the applicability of our approach to a wider range of NAS paradigms.

VI. Conclusion

We propose an MLO-based framework where a tester model generates adversarial validation examples to measure the worst-case performance of a learner model. A learner model improves the worst-case performance of its architecture by minimizing the loss on the generated adversarial validation data. Experiments on various datasets demonstrate the effectiveness of our method.

References

[1] Kaggle. [Online]. Available: https://www.kaggle.com/datasets/sartajbhuvaji/brain-tumor-classification-mri/versions/2
[2] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proc. 26th Annu. Int. Conf. Mach. Learn.*, 2009, pp. 41–48.
[3] A. Brock, J. Donahue, and K. Simonyan, "Large scale GAN training for high fidelity natural image synthesis," 2018, *arXiv:180911096*.
[4] H. Cai, L. Zhu, and S. Han, "ProxylessNAS: Direct neural architecture search on target task and hardware," in *Proc. ICLR*, 2019.
[5] X. Chen and C.-J. Hsieh, "Stabilizing differentiable architecture search via perturbation-based regularization," in *Proc. Int. Conf. Mach. Learn. (PMLR)*, 2020, pp. 1554–1565.
[6] X. Chen, R. Wang, M. Cheng, X. Tang, and C.-J. Hsieh, "DRNAS: Dirichlet neural architecture search," 2020, *arXiv:2006.10355*.
[7] X. Chen, L. Xie, J. Wu, and Q. Tian, "Progressive differentiable architecture search: Bridging the depth gap between search and evaluation," in *Proc. ICCV*, 2019.
[8] X. Chu, X. Wang, B. Zhang, S. Lu, X. Wei, and J. Yan, "DARTS-: robustly stepping out of performance collapse without indicators," 2020, *arXiv:2009.01027*.
[9] X. Chu, T. Zhou, B. Zhang, and J. Li, "Fair DARTS: eliminating unfair advantages in differentiable architecture search," 2019, *arXiv:1911.12126*.
[10] M. Cogswell, F. Ahmed, R. Girshick, L. Zitnick, and D. Batra, "Reducing overfitting in deep networks by decorrelating representations," 2015, *arXiv:1511.06068*.
[11] S. Dempe and A. Zemkoho, "Bilevel optimization," in *Springer Optimization and Its Applications*, vol. 161. Springer, 2020.
[12] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Piscataway, NJ, USA: IEEE Press, 2009, pp. 248–255.
[13] X. Dong and Y. Yang, "Searching for a robust neural architecture in four GPU hours," in *Proc. CVPR*, 2019.
[14] J. C. Duchi and H. Namkoong, "Learning models with uniform performance via distributionally robust optimization," *Ann. Statist.*, vol. 49, no. 3, pp. 1378–1406, 2021.
[15] Y. Ganin and V. Lempitsky, "Unsupervised domain adaptation by backpropagation," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1180–1189.
[16] S. Ghadimi and M. Wang, "Approximation methods for bilevel programming," 2018, *arXiv:1802.0f2246*.
[17] I. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 2672–2680.
[18] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.
[19] A. Graves, M. G. Bellemare, J. Menick, R. Munos, and K. Kavukcuoglu, "Automated curriculum learning for neural networks," in *Proc. Int. Conf. Mach. Learn. (PMLR)*, 2017, pp. 1311–1320.
[20] R. Grazzi, L. Franceschi, M. Pontil, and S. Salzo, "On the iteration complexity of hypergradient computation," in *Proc. Int. Conf. Mach. Learn. (PMLR)*, 2020, pp. 3748–3758.
[21] Y.-C. Gu et al., "Dots: Decoupling operation and topology in differentiable architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 12311–12320.
[22] C. He, H. Ye, L. Shen, and T. Zhang, "MiLeNAS: Efficient neural architecture search via mixed-level reformulation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020.
[23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016.
[24] D. Hendrycks et al., "The many faces of robustness: A critical analysis of out-of-distribution generalization," in *Proc. CVPR*, 2021.

[25] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," 2019, *arXiv:1903.12261*.

[26] D. Hendrycks, K. Zhao, S. Basart, J. Steinhardt, and D. Song, "Natural adversarial examples," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 15262–15271.

[27] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017.

[28] W. Hong et al., "DropNAS: Grouped operation dropout for differentiable architecture search," in *Proc. IJCAI*, 2020.

[29] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

[30] S. Hu et al., "DSNAS: Direct neural architecture search without parameter retraining," in *Proc. CVPR*, 2020.

[31] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. CVPR*, 2017.

[32] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel-Softmax," 2016, *arXiv:1611.01144*.

[33] K. Ji, J. Yang, and Y. Liang, "Bilevel optimization: Convergence analysis and enhanced design," in *Proc. Int. Conf. Mach. Learn. (PMLR)*, 2021, pp. 4882–4892.

[34] L. Jiang, D. Meng, S.-I. Yu, Z. Lan, S. Shan, and A. Hauptmann, "Self-paced learning with diversity," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, 2014, pp. 2078–2086.

[35] L. Jiang, D. Meng, Q. Zhao, S. Shan, and A. Hauptmann, "Self-paced curriculum learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 29, 2015.

[36] D. S. Kermany et al., "Identifying medical diagnoses and treatable diseases by image-based deep learning," *Cell*, vol. 172, no. 5, pp. 1122–1131, 2018.

[37] A. Krizhevsky and G. Hinton, "Convolutional deep belief networks on cifar-10," *Unpublished Manuscript*, vol. 40, no. 7, pp. 1–9, 2010.

[38] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," 2009.

[39] M. Pawan Kumar, B. Packer, and D. Koller, "Self-paced learning for latent variable models," in *Proc. Adv. Neural Inf. Process. Syst.*, pp. 1189–1197, 2010.

[40] Y. J. Lee and K. Grauman, "Learning the easy things first: Self-paced visual category discovery," in *Proc. CVPR*, 2011.

[41] L. Li, M. Khodak, M.-F. Balcan, and A. Talwalkar, "Geometry-aware gradient algorithms for neural architecture search," 2021, *arXiv:2004.07802*.

[42] Y. Li, G. Hu, Y. Wang, T. Hospedales, N. M. Robertson, and Y. Yang, "DADA: Differentiable automatic data augmentation," 2020, *arXiv:2003.03780*.

[43] C. Liu et al., "Progressive neural architecture search," in *Proc. ECCV*, 2018.

[44] H. Liu, K. Simonyan, O. Vinyals, C. Fernando, and K. Kavukcuoglu, "Hierarchical representations for efficient architecture search," in *Proc. ICLR*, 2018.

[45] H. Liu, K. Simonyan, and Y. Yang, "DARTS: differentiable architecture search," in *Proc. ICLR*, 2019.

[46] R. Liu, Y. Liu, S. Zeng, and J. Zhang, "Towards gradient-based bilevel optimization with non-convex followers and beyond," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021.

[47] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: practical guidelines for efficient CNN architecture design," in *Proc. ECCV*, 2018.

[48] T. Matiisen, A. Oliver, T. Cohen, and J. Schulman, "Teacher-student curriculum learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 9, pp. 3732–3740, Sep. 2020.

[49] C. Mayer and R. Timofte, "Adversarial sampling for active learning," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2020, pp. 3071–3079.

[50] M. Mirza and S. Osindero, "Conditional generative adversarial nets," 2014, *arXiv:1411.1784*.

[51] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, "Spectral normalization for generative adversarial networks," 2018, *arXiv:1802.05957*.

[52] H. Pham, M. Y. Guan, B. Zoph, Q. V. Le, and J. Dean, "Efficient neural architecture search via parameter sharing," in *Proc. ICML*, 2018.

[53] E. A. Platanios, O. Stretcu, G. Neubig, B. Poczos, and T. M. Mitchell, "Competence-based curriculum learning for neural machine translation," 2019, *arXiv:1903.09848*.

[54] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 4780–4789.

[55] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved techniques for training GANs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016.

[56] R. Sato, M. Tanaka, and A. Takeda, "A gradient method for multilevel optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 7522–7533, 2021.

[57] M. Shu, C. Liu, W. Qiu, and A. Yuille, "Identifying model weakness with adversarial examiner," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 7, 2020, pp. 11998–12006.

[58] A. Sinha, H. Namkoong, R. Volpi, and J. Duchi, "Certifying some distributional robustness with principled adversarial training," 2017, *arXiv:1710.10571*.

[59] S. Sinha, H. Zhang, A. Goyal, Y. Bengio, H. Larochelle, and A. Odena, "Small-GAN: Speeding up GAN training using core-sets," in *Proc. Int. Conf. Mach. Learn. (PMLR)*, 2020, pp. 9005–9015.

[60] S. Sinha, Z. Zhao, A. Goyal, A. Parth Goyal, C. A. Raffel, and A. Odena, "Top-k training of GANs: Improving GAN performance by throwing away bad samples," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 14638–14649.

[61] V. I. Spitkovsky, H. Alshawi, and D. Jurafsky, "From baby steps to leapfrog: How "less is more" in unsupervised dependency parsing," in *Proc. NAACL*, 2010.

[62] F. P. Such, A. Rawal, J. Lehman, K. O. Stanley, and J. Clune, "Generative teaching networks: Accelerating neural architecture search by learning to generate synthetic training data," 2019, *arXiv:1912.07768*.

[63] Z. Sun et al., "AGNAS: Attention-guided micro and macro-architecture search," in *Proc. ICML*, 2022.

[64] C. Szegedy et al., "Going deeper with convolutions," in *Proc. CVPR*, 2015.

[65] Y. Tsvetkov, M. Faruqui, W. Ling, B. MacWhinney, and C. Dyer, "Learning the curriculum with Bayesian optimization for task-specific word representation learning," 2016, *arXiv:1605.03852*.

[66] R. Wang, M. Cheng, X. Chen, X. Tang, and C.-J. Hsieh, "Rethinking architecture selection in differentiable NAS," 2021, *arXiv:2108.04392*.

[67] X. Wang, A. Jabri, and A. A. Efros, "Learning correspondence from the cycle-consistency of time," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 2566–2576.

[68] S. Xie, H. Zheng, C. Liu, and L. Lin, "SNAS: Stochastic neural architecture search," in *Proc. ICLR*, 2019.

[69] Y. Xu et al., "PC-DARTS: partial channel connections for memory-efficient architecture search," in *Proc. ICLR*, 2020.

[70] Y. Xue, X. Han, F. Neri, J. Qin, and D. Pelusi, "A gradient-guided evolutionary neural architecture search," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 3, pp. 4345–4357, Mar. 2025.

[71] J. Yang, K. Ji, and Y. Liang, "Provably faster algorithms for bilevel optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021.

[72] Y. Yang, H. Li, S. You, F. Wang, C. Qian, and Z. Lin, "ISTA-NAS: Efficient and consistent neural architecture search by sparse coding," 2020.

[73] P. Ye, B. Li, Y. Li, T. Chen, J. Fan, and W. Ouyang, " B-DARTS: Beta-decay regularization for differentiable architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10874–10883.

[74] L. Yu, W. Zhang, J. Wang, and Y. Yu, "SeqGAN: Sequence generative adversarial nets with policy gradient," in *Proc. AAAI*, 2017.

[75] A. Zela, T. Elsken, T. Saikia, Y. Marrakchi, T. Brox, and F. Hutter, "Understanding and robustifying differentiable architecture search," 2019, *arXiv:1909.09656*.

[76] X. Zhang, Y. Li, X. Zhang, Y. Wang, and J. Sun, "Differentiable architecture search with random features," in *Proc. CVPR*, 2023.

[77] Y. Zhao, C. Li, P. Yu, J. Gao, and C. Chen, "Feature quantization improves GaN training," 2020, *arXiv:2004.02088*.

[78] P. Zhou, C. Xiong, R. Socher, and S. C. H. Hoi, "Theory-inspired path-regularized differential network architecture search," 2020, *arXiv:2006.16537*.

[79] T. Zhou, S. Wang, and J. Bilmes, "Curriculum learning by optimizing learning dynamics," in *Proc. AISTATS*, 2021.

[80] T. Zhou, S. Wang, and J. Bilmes, "Curriculum learning by dynamic instance hardness," in *Proc. NeurIPS*, 2020.

[81] Y. Zhou, B. Yang, D. F. Wong, Y. Wan, and L. S. Chao, "Uncertainty-aware curriculum learning for neural machine translation," in *Proc. ACL*, 2020.

[82] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *Proc. ICLR*, 2017.

[83] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. CVPR*, 2018.

Haochen Zhang received the B.E and M.E degree in electrical engineering from the University of Science and Technology of China, Hefei, China, in 2017 and 2020, respectively. He is currently working toward the Ph.D. degree in electrical and computer engineering with the University of California at San Diego, San Diego, CA, USA.

His research interests include computer vision on medical image analysis, especially retinal images.

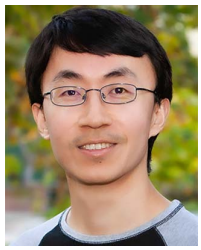
Xuefeng Du received the B.E. degree in electronic engineering from Xi'an Jiaotong University (XJTU), Xi'an, China, and the Ph.D. degree in computer sciences from the University of Wisconsin-Madison (UW-Madison), Madison, WI, USA, in 2020.

He is currently an Assistant Professor with the College of Computing and Data Science (CCDS), Nanyang Technological University, Singapore.

Ruisi Zhang received the bachelor's degree in electronic engineering from Shanghai Jiao Tong University, Shanghai, China, in 2021. She is currently working toward the Ph.D. degree in machine learning with UC San Diego, La Jolla, CA, USA.

Her research interests include large language model efficiency, distributed training acceleration, and algorithm–system codesign for scalable and secure LLM applications.

Dr. Zhang was recognized as the 2025 Machine Learning and Systems Rising Star.



Pengtao Xie (Member, IEEE) received the Ph.D. degree in machine learning from the Machine Learning Department, Carnegie Mellon University, Pittsburgh, PA, USA, in 2018.

He is an Associate Professor with the Department of Electrical and Computer Engineering, Department of Molecular Biology, Department of Bioengineering, Department of Medicine, and School of Pharmacy and Pharmaceutical Sciences, University of California San Diego, La Jolla, CA, USA. His research interests include machine learning for biology, medicine, and healthcare.

Prof. Xie was recognized with NSF Career Award, National Institutes of Health Maximizing Investigators' Research Award (NIH MIRA) Award, Global Top-100 Chinese Young Scholars in AI by Baidu, top-5 finalist for the AMIA Doctoral Dissertation Award, International Conference on Learning Representations (ICLR) Notable-Top-5% paper, Amazon Amazon Web Services (AWS) Machine Learning Research Award, Tencent AI-Lab Faculty Award, Innovator Award by the Pittsburgh Business Times, among others. He serves as an Associate Editor for the *ACM Transactions on Computing for Healthcare*, Senior Area Chairs for International Conference on Machine Learning (ICML) and The Association for the Advancement of Artificial Intelligence (AAAI), Area Chair for *Neural Information Processing Systems*, etc.